Heap và hàng đợi ưu tiên

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 5: Ưu tiên và đồ thị.
- Các bài trong chương: Heap và hàng đợi ưu tiên; Đồ thị, BFS và DFS.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Mô phỏng min-heap và phân biệt ưu tiên với FIFO.

Kiến thức nền

Hàng đợi ưu tiên lấy phần tử theo độ ưu tiên, không nhất thiết theo thứ tự đến. Min-heap nhị phân thường biểu diễn bằng mảng và hình dạng cây nhị phân hoàn chỉnh; mỗi nút cha có khóa không lớn hơn các con. Phần tử nhỏ nhất ở gốc nhưng toàn bộ mảng chưa được sắp xếp.

Với chỉ số bắt đầu từ 0, con trái của i ở 2*i+1, con phải ở 2*i+2. Chèn phần tử vào cuối rồi đưa lên để khôi phục tính heap; lấy min thay gốc bằng phần tử cuối rồi đưa xuống. Chiều cao O(log n) dẫn tới chi phí chèn/lấy min O(log n); xem min O(1).

Heap hữu ích khi liên tục cần công việc ưu tiên hoặc k phần tử nổi bật. Nếu nhiều việc bằng ưu tiên và cần giữ thứ tự đến, bổ sung bộ đếm thứ tự. Không dùng heap để thay thế mọi yêu cầu tìm kiếm theo khóa tùy ý.

1. Bất biến và chỉ số

Min-heap yêu cầu cha không lớn hơn các con, không yêu cầu toàn nhánh trái nhỏ hơn toàn nhánh phải như BST. Hình dạng hoàn chỉnh giúp biểu diễn bằng mảng. Với i>0, cha ở (i-1)//2; con ở 2*i+1 và 2*i+2 nếu các chỉ số còn trong mảng.

2. Cài đặt sử dụng thư viện chuẩn

```
import heapq

heap = []
for value in [5, 2, 8, 1]:
    heapq.heappush(heap, value)
```

```
output = [heapq.heappop(heap) for _ in range(len(heap))]
assert output == [1, 2, 5, 8]
```

Đây là minh họa API min-heap đã có từ lâu, không phụ thuộc các hàm max-heap mới. Không gọi heappop trên heap rỗng nếu chưa quy định xử lý lỗi.

3. Truy vết thao tác và độ phức tạp

Thêm 1 vào [2,5,8] đặt ở cuối rồi đổi với cha 5, sau đó với cha 2. Lấy min thay gốc bằng phần tử cuối, đưa xuống qua con nhỏ hơn để khôi phục bất biến. Chiều cao là O(log n). heapify cả mảng có thể O(n); không nhân n với log n nếu đang phân tích thuật toán heapify tuyến tính.

4. Ưu tiên bằng nhau và ứng dụng

Lưu tuple (priority, sequence, task) để giữ thứ tự đến khi priority bằng nhau và tránh so sánh trực tiếp task không có thứ tự. Cần nêu quy ước số nhỏ là ưu tiên cao hay ngược lại.

Thực hành: thêm ba việc có ưu tiên 2,1,2; mong đợi việc ưu tiên 1 trước rồi hai việc ưu tiên 2 theo sequence. Giải thích tại sao heap không tối ưu mọi yêu cầu tìm theo mã task tùy ý.

> **Bài tập vận dụng bổ sung**
>
> Mô phỏng thêm 1 vào heap [2,5,8]. Lập lịch ba công việc có ưu tiên khác nhau và giải thích quy ước khi bằng ưu tiên.

Hướng giải bài tập bổ sung

Kết quả lấy lần lượt là 1,2,5,8 nếu không thêm phần tử khác. Nêu rõ số ưu tiên nhỏ hay lớn được phục vụ trước.

Câu hỏi tự kiểm tra

1. Gốc min-heap chứa gì?
2. Mảng biểu diễn heap có luôn sắp xếp tăng dần không?
3. Con trái của chỉ số i trong heap mảng 0-based ở đâu?
4. Chèn vào heap nhị phân thường có chi phí nào?
5. Priority queue khác queue FIFO ở điểm nào?

Gợi ý đáp án và điểm cần nhớ

- Khóa nhỏ nhất. Quy tắc heap đảm bảo min tại gốc.
- Không. Quy tắc cha–con yếu hơn thứ tự tuyến tính toàn mảng.
- 2*i+1. Công thức xuất phát từ biểu diễn cây theo từng mức.
- O(log n). Phần tử có thể đi qua chiều cao heap.
- Lấy theo ưu tiên thay vì bắt buộc thứ tự đến. Tiêu chí lấy phần tử là độ ưu tiên.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html